

## **Reflection Report: ShopEasy Test Suite**

**Name and Surname: Kevser Katırcıoğlu**

**Student Number: 220717014**

During the development of the test suite for the ShopEasy system, applying different testing techniques revealed distinct issues in the starter code. Structural testing revealed that the `applyDiscount` method in the `ShoppingCart` class is completely broken; it fails to apply the discount to the total price and simply returns the original amount. Later, property-based testing with `jqwik` uncovered a much more hidden edge case regarding cart commutativity. The tool generated two `Product` instances with the exact same ID but different prices. Because the cart logic only checks the product ID to update the quantity and ignores the new price, inserting product A then B resulted in a different total than inserting B then A.

When considering which technique was most effective per unit of effort, I believe structural testing combined with JaCoCo coverage was the most productive. While property-based testing caught a deeper logic flaw, setting up the custom data providers and diagnosing the root cause took a significant amount of time. Structural testing, on the other hand, gave me immediate visual feedback on exactly which branches of code were missed. This allowed me to write targeted test cases very quickly and achieve high coverage with relatively low effort.

Based on the testing pyramid discussed in Chapter 1, I would classify our current test suite entirely as a unit testing suite. We tested isolated components like `PriceCalculator` and `ShoppingCart` independently. Even for the `OrderProcessor`, we used `Mockito` to isolate it from external systems. What is completely missing from this suite are integration tests and end-to-end (E2E) tests. We don't have any tests verifying if `OrderProcessor` successfully communicates with a real database or a live `InventoryService`.

Mocking the dependencies for OrderProcessor allowed me to test the core order logic without making real network calls or accidentally triggering actual financial transactions through a payment gateway, which is essential for a fast and safe test environment. However, mocking prevents me from testing the actual integration. If the real PaymentGateway changes its API behavior or response times, my mocked tests will still pass, giving a false sense of security. Mocking becomes a bad idea when we start mocking internal domain objects (like mocking the ShoppingCart inside the processor) because it ties the tests too closely to implementation details rather than observable behavior.

If I had one more week to work on this project, I would focus on writing integration tests for the mocked services and adding concurrency testing. In a real e-commerce system, multiple users or threads might attempt to add items to the same cart or check out simultaneously. Our current unit tests are strictly single-threaded and ignore race conditions.

**Bonus: PIT Mutation Testing Analysis** During the mutation testing, I analyzed two mutants that survived. The first mutant (Line 66) changed a conditional boundary. It survived because our structural tests only covered the happy path with a positive quantity, but we did not test the exact boundary value of 0. We can kill this mutant by adding a specific boundary test (`assertThatThrownBy(() -> cart.updateQuantity(apple.getId(), 0))`) to ensure it throws an `IllegalArgumentException`. The second mutant (Line 92) replaced double subtraction with addition inside the `applyDiscount` logic. It survived because the original method has an intentional bug and doesn't actually alter the total. Since our test asserts the unaltered total just to pass the build, changing the math operation doesn't break the test. We cannot kill this mutant until the actual bug in the production code is fixed, which would then allow us to assert the correct discounted value and catch any altered math operations.